Customer Behavior Analysis and Churn Prediction in E-Commerce Using BI and ML

Done By:

Mariam Luay Alhusseini

Student Id

202210921

Submitted to Business Intelligence & Data Analysis Department

University of Petra

In Partial Fulfillment of the Requirements

For the Degree of Business Intelligence & Data Analysis

Acknowledgement

I would like to express my deepest gratitude to my professors and academic mentors for their invaluable guidance, encouragement, and thoughtful insights throughout the course of this project. Their continuous support has not only enhanced the quality of this work but has also contributed significantly to my academic growth and development as an aspiring data professional. Their constructive feedback and unwavering belief in my potential have been instrumental in refining my analytical skills and fostering a mindset geared toward innovation and lifelong learning.

My sincere appreciation extends to the dedicated faculty and staff of the institution, whose commitment to academic excellence created an environment that fostered curiosity, critical thinking, and a passion for research. Their readiness to assist, coupled with their depth of knowledge and professionalism, greatly facilitated the successful completion of this project.

To my beloved mother and father, thank you for your unconditional love, immense sacrifices, and constant prayers. Your belief in me has been the foundation of my perseverance and the source of my strength throughout every stage of this journey. Your support and values have shaped the person I am today and the one I strive to become.

To my family and friends, your patience, motivation, and continuous encouragement gave me the resilience to overcome challenges and celebrate milestones. Thank you for being my unwavering support system, cheering me on during long nights and reminding me of my goals in times of doubt.

Finally, I am sincerely grateful to everyone who contributed to this journey, directly or indirectly. Whether through a word of encouragement, a shared resource, or silent support, each gesture has left a meaningful impact. I carry this collective support with me, and it will remain a treasured part of my academic and personal growth

Abstract

In the rapidly evolving landscape of e-commerce, customer retention has become a cornerstone of sustainable business growth. This project aims to uncover actionable insights into customer behavior and predict churn using a comprehensive data science approach that blends Business Intelligence (BI), descriptive analytics, and machine learning (ML) techniques. The study begins with meticulous data preprocessing and cleansing to ensure data quality and integrity, followed by exploratory data analysis (EDA) to identify key behavioral trends and relationships among variables such as tenure, order frequency, cashback usage, coupon redemption, and complaints.

Statistical hypothesis testing is conducted to validate the significance of these variables, leading to the selection of features most relevant to predicting customer churn. Two classification models—Logistic Regression and Random Forest—are developed, trained, and evaluated using key performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. The Random Forest model exhibited higher performance, especially in identifying at-risk customers, making it a valuable tool for targeted retention efforts.

Additionally, the project employs unsupervised learning via K-Means clustering to segment the customer base into meaningful groups based on purchasing behavior and engagement patterns. These segments—such as high-value loyal customers, budget-conscious users, and complaint-prone churn risks—allow businesses to tailor their marketing and service strategies effectively. The findings are visualized through interactive BI dashboards, enabling business stakeholders to interpret complex data intuitively and make informed decisions.

Ultimately, this project demonstrates how combining predictive modeling with customer segmentation and BI visualization can empower e-commerce platforms to enhance user engagement, reduce churn, and drive long-term profitability. The methodologies applied offer a replicable framework for data-driven decision-making in the digital commerce domain.

Contents

Acknowledgement 2
Abstract 3
Introduction 5
Problem Statement 6
Goals & Objectives 6
Literature Review / Theoretical Framework 7
Methodology 8
Model Selection & Evaluation 9
Evaluation of Methods 9
Data Understanding 10
- Columns Description: 11

Data Cleansing 13
Descriptive Statistics & EDA 19
- Null Hypotheses 31
  - T-Test: OrderCount Between Churned and Retained Customers 32
  - T-Test: CashbackAmount Between Churned and Retained 33
  - Pearson Correlation: CouponUsed vs. OrderCount 33
  - Chi-Square Test: Complain vs. Churn 33

Predictive Analysis 34
- Logistic Regression/ Random Forest 34
- Clustering 41
  - Cluster 3: High Spenders 44
  - Cluster 2: Occasional / Low Engagement Users 44
  - Cluster 1: Budget Shoppers 44
  - Cluster 0: Complaint-Prone Users 45

Power BI Dashboards 45
- Customer Behavior & Sales Dashboard 46
- Cluster 3: High Spenders – Customer Behavior & Insights 47
- Cluster 2: Occasional Users – Low Engagement Profile 48
- Cluster 1: Budget Shoppers – Segment Analysis 49
- Cluster 0: Complaint-Prone Users – Behavior & Risk Insights 50

Quantitative Assessment 51
Key Findings 53
Conclusion: 53
References 54

Introduction

In the fast-paced world of e-commerce, retaining customers and understanding their behavior are among the most critical factors for achieving long-term success. With rising customer acquisition costs and increasing competition, businesses are now focusing more than ever on customer retention, satisfaction, and targeted engagement. A data-driven understanding of customer behavior enables companies to design effective marketing strategies, predict churn risk, and personalize user experiences.

This project presents a comprehensive analysis of customer data in an e-commerce setting, with the goal of uncovering patterns, predicting churn, and segmenting users based on their behavioral and transactional profiles. By integrating Business Intelligence (BI) tools with statistical and machine learning (ML) techniques, we aim to convert raw data into meaningful insights that directly support data-informed decision-making and enhance customer lifecycle management.

The dataset used in this study was sourced from Kaggle and comprised a wide range of customer attributes, including tenure, order count, cashback amount, hours spent on the mobile application, coupon usage, preferred payment method, complaint history, and churn labels. Our approach begins with thorough data preprocessing and cleansing to ensure reliability and consistency, followed by exploration data analysis (EDA) to reveal underlying behavioral trends and identify significant patterns in customer activity.

To validate these relationships, we apply hypothesis testing to highlight statistically significant factors influencing churn. These insights guide the selection of predictive features used to train classification models—Logistic Regression and Random Forest—allowing us to evaluate churn risk based on behavioral indicators. In addition to supervised learning, the project employs unsupervised clustering techniques (e.g., K-Means) to segment customers into meaningful groups such as loyal high-value users, deal seekers, and potential churners. These clusters form the basis for tailored marketing and customer retention strategies.

Interactive dashboards are also developed to visualize insights in a user-friendly and accessible manner, bridging the gap between complex analytics and business interpretation. These dashboards empower stakeholders to monitor customer behavior dynamically and act on churn warnings in real time.
The ultimate objective of this project is to establish a scalable, data-informed framework for customer analytics in the e-commerce domain. By leveraging predictive modeling, behavioral segmentation, and business intelligence, this study provides actionable strategies for reducing churn, enhancing engagement, and fostering long-term customer loyalty. Ultimately, this project showcases the transformative impact of applied data science in modern business environments.

Problem Statement

In the highly competitive e-commerce industry, businesses face a significant challenge in retaining customers and preventing churn. Many companies invest heavily in acquiring new users but often lack a clear understanding of why customers disengage or fail to return. Without timely insights into customer behavior, organizations risk losing valuable clients, missing marketing opportunities, and suffering financial losses due to avoidable churn.

The problem this project addresses is twofold:

1. There is the absence of a reliable, data-driven method to predict which customers are at risk of churning.
2. The lack of effective customer segmentation strategies to personalize offers and improve user engagement based on behavioral traits.

This project aims to tackle these challenges by analyzing customer activity data using statistical and machine learning techniques. By identifying patterns related to churn and grouping customers based on shared behaviors, the goal is to enable smarter decision-making, proactive retention strategies, and more effective marketing campaigns.

Goals & Objectives

Goal:

To analyze customer behavior data from an e-commerce platform to predict customer churn, segment users based on behavioral patterns and generate actionable insights that support data-driven business decisions.

Objectives:

- Identify patterns and key factors influencing customer churn in an e-commerce business.
- Build predictive models to accurately forecast which customers are likely to churn.
- Segment customers into meaningful groups (clusters) for targeted strategies.
- Develop actionable insights to improve customer retention and satisfaction.
- Create interactive Power BI dashboards for real-time analysis and decision-making.
- Enable data-driven strategies for marketing, service improvement, and risk reduction.

Literature Review / Theoretical Framework

Customer churn has emerged as a critical focus in the field of customer relationship management, particularly within the e-commerce sector. It directly influences revenue, customer lifetime value, and marketing costs. Traditional customer tracking systems often fail to detect early warning signs of churn or to personalize retention strategies effectively. This has led to an increasing reliance on Business Intelligence (BI) tools and machine learning (ML) algorithms to support proactive, data-driven decision-making.

Several studies have underscored the significance of predictive analytics in understanding and preventing customer churn. Idris, Khan, and Lee (2012) demonstrated that classification models such as logistic regression and random forest can achieve high performance in predicting churn in telecom and e-commerce datasets. Their work also emphasized the role of behavioral features —such as purchase frequency, inactivity duration, and complaint rating identifying customers at risk of leaving.

Marr (2017) highlighted the growing role of BI dashboards and real-time analytics in customer experience management. He argued that interactive dashboards not only enhance operational awareness but also serve as essential tools for tracking churn metrics and delivering insights to decision-makers. This reinforces the project's emphasis on integrating BI for visualization and monitoring of key performance indicators (KPIs).

In terms of segmentation, Kumar and Reinartz (2016) discussed the importance of customer profiling and cluster-based marketing. They advocated for the use of unsupervised learning techniques—such as K-Means and Hierarchical Clustering typing high-value, price-sensitive, or complaint-prone customer groups. This approach supports the project's goal of dividing customers into actionable segments to enable more targeted retention efforts.

Sahin et al. (2013) explored the impact of customer satisfaction, loyalty programs, and service quality on churn rates, reinforcing the importance of variables such as cashback and coupon usage in predictive modeling. Their research aligns with the hypothesis testing performed in this project, which confirms that incentives and engagement frequency are statistically linked to customer retention.

Finally, Tsai and Lu (2009) examined the effectiveness of combining descriptive statistics with machine learning in customer analytics. Their results validated those preprocessing techniques, such as outlier removal and normalization, significantly improving model accuracy and reliability. This supports the methodological design of the current study, which integrates data cleansing, exploratory analysis, and feature engineering before modeling.

This theoretical foundation affirms the effectiveness of combining BI dashboards, supervised learning, statistical validation, and clustering algorithms in developing intelligent customer management systems. The project builds on this multidisciplinary framework to help businesses detect churn risk, segment customers effectively, and implement more strategic, data-informed engagement policies

Methodology

This project follows a structured data science pipeline combining business intelligence, statistical analysis, and machine learning to analyze customer behavior, predict churn, and segment users for strategic marketing purposes. The methodology consists of the following key phases:

1. Data Collection The dataset was sourced from Kaggle and represents a simulated e-commerce platform with over 5,000 unique customer records. It contains both numerical and categorical features, including variables such as Tenure, OrderCount, CashbackAmount, CouponUsed, HourSpendOnApp, Complain, PreferredPaymentMode, and Churn status (the target variable).
2. Data Cleansing and Preprocessing To ensure the reliability of results, a thorough data cleansing process was applied: Handling Missing Values: Median or mean imputation was used based on distribution type. Removing Duplicates: Identified and removed duplicate records.

Outlier Detection: IQR-based filtering and boxplots were used to detect and optionally remove extreme values. Formatting Categorical Variables: Standardized inconsistent entries (e.g., payment modes, gender) to ensure uniformity. Derived features were also created to enrich the dataset:

CashbackPerOrder = CashbackAmount / (OrderCount + 1)

AppUsageIntensity = HourSpendOnApp / (Tenure + 1)

3. Exploratory Data Analysis (EDA) Descriptive statistics (mean, median, standard deviation) and visualizations (histograms, boxplots, violin plots) were used to summarize the data and explore variable distributions. Correlation matrices and heatmaps were generated to assess linear relationships between numerical features and to aid in feature selection. Pivot tables and bar plots were used to analyze customer behavior across tenure groups, complaint levels, and city tiers.
4. Hypothesis Testing Statistical hypothesis testing was conducted to validate assumptions regarding key relationships: T-tests to compare OrderCount and CashbackAmount between churned vs. non- churned customers. Chi-Square test to examine the relationship between complaints and churn.

Pearson correlation tests to explore associations among variables like CouponUsed, HourSpendOnApp, and OrderCount. These tests informed the feature selection process for modeling.

5. Churn Prediction Modeling Two machine learning algorithms were implemented: Logistic Regression (baseline, interpretable) Random Forest Classifier (robust, non-linear) Steps included: Feature scaling using StandardScaler (for Logistic Regression) Train-test split (80/20) Model evaluation using accuracy, precision, recall, F1-score, ROC-AUC

Visualizations: Confusion matrix, ROC curve

Both models were compared to assess their predictive performance, and feature importance was extracted from the Random Forest model. 2. Customer Segmentation Unsupervised learning was used to cluster customers into behavior-based segments: K-Means clustering was applied using scaled features such as OrderCount, CashbackAmount, CouponUsed, and Complain. The optimal number of clusters was determined using the Elbow Method.

Each cluster was profiled based on average behaviors to define groups such as High Spenders, Budget Shoppers, and Complaint-Prone Users. PCA was used for 2D visualization of the clusters.

6. BI Dashboard Integration To communicate findings, BI dashboards were proposed using Power BI or Tableau. These included: Churn Risk Overview Revenue Trends by Segment Behavioral KPIs by Cluster Top Cities and Product Categories

   These dashboards allow stakeholders to explore the data dynamically and support decision-making across marketing and operations.

Model Selection & Evaluation

Supervised Models

Two classification models were used to predict customer churn:

- Logistic Regression – chosen for its simplicity and interpretability. - Random Forest – selected for its ability to handle complex patterns and provide high accuracy.

Both models were trained on key features such as Tenure, OrderCount, CashbackAmount, CouponUsed, HourSpendOnApp, and Complain. The dataset was split into training and testing sets (80/20), and StandardScaler was applied for Logistic Regression. Models were evaluated using Accuracy, Precision, Recall, F1 Score, ROC-AUC, Confusion Matrix

- Random Forest outperformed Logistic Regression in most metrics, especially in recall and AUC, making it the more suitable model for predicting churn. Logistic Regression, however, provided valuable interpretability and feature insights. Both models together support a balanced approach: performance and understanding.

Evaluation of Methods

In this project, Logistic Regression achieved an accuracy of 82%, precision of 63%, recall of 25%, F1 score of 36%, and ROC AUC of 61%. In comparison, the Random Forest model performed better with 87% accuracy, 73% precision, 55% recall, F1 score of 63%, and ROC AUC of 75%. These results indicate that Random Forest outperformed Logistic Regression across all evaluation metrics and is more suitable for predicting customer churn in this case.

Data Understanding

is the process of getting familiar with the data you have, what it looks like, what it contains, and what it means—so you can use it effectively for analysis or building models. This step usually involves exploring the data, checking its quality, identifying patterns, and figuring out any issues like missing values or errors.

```
In [6]: from google.colab import drive
        drive.mount('/content/drive')

        import pandas as pd

        path = '/content/drive/MyDrive/E Commerce Dataset.xlsx'
        data = pd.read_excel(path, index_col='CustomerID')
```

This code connects Google Colab environment to Google Drive using drive.mount, allowing to access files stored there. It then imports the pandas library, which is commonly used for data analysis.

The Excel file named "E Commerce Dataset.xlsx" is read from your Drive using its file path, and it's loaded into a DataFrame called data. While loading, the column 'CustomerID' is set as the index to uniquely identify each row. This setup enables you to begin analyzing and manipulating your dataset directly within the Colab notebook.

```
In [7]: data.head()
```

		Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpendOnApp	NumberOfDeviceRegistered
CustomerID										
50001		1	4.0	Mobile Phone	3	6.0	Debit Card	Female	3.0	3
50002		1	NaN	Phone	1	8.0	UPI	Male	3.0	4
50003		1	NaN	Phone	1	30.0	Debit Card	Male	2.0	4
50004		1	0.0	Phone	3	15.0	Debit Card	Male	2.0	4
50005		1	0.0	Phone	1	12.0	CC	Male	NaN	3

```
In [8]: print("Number of Rows: ", len(data))
        print("Number of Columns: ", len(data.columns))
        print("Table Size: ", data.size)
```

```
Number of Rows:  5630
Number of Columns:  19
Table Size:  106970
```

Columns Description:

1. CustomerID

- Description: Unique identifier for each customer
- Use: Used for reference only (not included in modeling)

2. Tenure

- Description: Number of months the customer has been active
- Use: Used in EDA, hypothesis testing, feature engineering, modeling, and clustering.

3. WarehouseToHome

- Description: Estimated delivery time (in minutes) from warehouse to home
- Use: Evaluated in cleansing and EDA; excluded from modeling

4. HourSpendOnApp

- Description: Average number of hours spent on the app per month.
- Use: Used in EDA, correlation analysis, feature engineering, modeling, and clustering

5. NumberOfDeviceRegistered

- Description: Total number of devices associated with the user's account
- Use: Explored in EDA; excluded from modeling

6. PreferedOrderCat

- Description: Customer's most frequently purchased product category
- Use: Used in BI dashboards and segmentation insights

7. SatisfactionScore

- Description: Self-reported satisfaction level (1–5 scale)
- Use: Used in descriptive analysis and segmentation

8. MaritalStatus

- Description: Marital status of the customer
- Use: Used in profiling and optional BI views

9. NumberOfAddress

- Description: Number of shipping addresses saved by the customer
- Use: Examined in EDA; excluded from model

10. Complaint

- Description: Whether the customer submitted a complaint (0 = No, 1 = Yes)
- Use: Used in hypothesis testing, modeling, and clustering

11. DaySinceLastOrder

- Description: Number of days since the customer's last purchase
- Use: Explored in EDA and cleansing

12. CashbackAmount

- Description: Total cashback amount received by the customer
- Use: Used in EDA, hypothesis testing, modeling, and clustering

13. CouponUsed

- Description: Number of coupons used by the customer
- Use: Used in statistical testing, modeling, and segmentation

14. OrderCount

- Description: Total number of orders placed by the customer
- Use: Used extensively in EDA, modeling, clustering, and BI visualizations

15. OrderAmountHikeFromlastYear

- Description: Percent increase in order amount compared to the previous year
- Use: Used in EDA and segmentation

16. PreferredPaymentMode

- Description: Customer's most used payment method
- Use: Used in BI dashboards and segmentation

17. Gender

- Description: Gender of the customer
- Use: Used in segmentation and visual profiling

18. CityTier

- Description: Indicates the customer's city classification (e.g., Tier 1, Tier 2, Tier 3), reflecting the size and development level of their location.
- Use: Used in descriptive analysis, pivot tables, and BI dashboards.

19. Churn

- Description: Target variable (1 = Churned, 0 = Retained)
- Use: Main output variable for prediction models

Data Cleansing

is the process of identifying and correcting (or removing) inaccurate, incomplete, duplicate, or irrelevant data from a dataset to improve its quality and reliability.

Key steps in data cleansing include:

- Removing duplicates
- Fixing structural errors (e.g., inconsistent formatting or typos)
- Filling in missing values or handling them appropriately
- Filtering out irrelevant data
- Validating and correcting data against a standard or rule

Purpose: The goal of data cleansing is to ensure that the data is accurate, consistent, and usable—especially important for analysis, reporting, and decision-making

```
In [9]: data.isnull().sum()
```

	0
Churn	0
Tenure	264
PreferredLoginDevice	0
CityTier	0
WarehouseToHome	251
PreferredPaymentMode	0
Gender	0
HourSpendOnApp	255
NumberOfDeviceRegistered	0
PreferedOrderCat	0
SatisfactionScore	0
MaritalStatus	0
NumberOfAddress	0
Complain	0
OrderAmountHikeFromlastYear	265
CouponUsed	256
OrderCount	258
DaySinceLastOrder	307
CashbackAmount	0

Purpose: To detect which features contain incomplete or missing data, critical first step in the data cleansing process and helps decide how to handle those missing values

```
In [10]:
data['Tenure'].fillna(data['Tenure'].median(), inplace=True)
data['WarehouseToHome'].fillna(data['WarehouseToHome'].mean(), inplace=True)
data['HourSpendOnApp'].fillna(data['HourSpendOnApp'].median(), inplace=True)
data['OrderAmountHikeFromlastYear'].fillna(data['OrderAmountHikeFromlastYear'].median(), inplace=True)
data['CouponUsed'].fillna(data['CouponUsed'].median(), inplace=True)
data['OrderCount'].fillna(data['OrderCount'].median(), inplace=True)
data['DaySinceLastOrder'].fillna(data['DaySinceLastOrder'].median(), inplace=True)
```

This code fills missing values in key columns to ensure the dataset is clean and ready for analysis.

- Median is used for columns like Tenure, OrderCount, and CouponUsed because these features are often skewed and the median reduces the effect of outliers.

- Mean is used for WarehouseToHome because it's likely to follow a normal distribution.

This step is essential to prevent errors in modeling and to keep the statistical accuracy of the dataset.

```
In [11]: data.isnull().sum()
```

	0
Churn	0
Tenure	0
PreferredLoginDevice	0
CityTier	0
WarehouseToHome	0
PreferredPaymentMode	0
Gender	0
HourSpendOnApp	0
NumberOfDeviceRegistered	0
PreferedOrderCat	0
SatisfactionScore	0
MaritalStatus	0
NumberOfAddress	0
Complain	0
OrderAmountHikeFromlastYear	0
CouponUsed	0
OrderCount	0
DaySinceLastOrder	0
CashbackAmount	0

verification step after data imputation to make sure the dataset is fully complete.

```
In [12]: data.duplicated().value_counts()
```

	count
False	5072
True	558

What it does:

Checks for duplicated rows in your dataset.

Returns how many rows are duplicates (True) vs. unique (False).

Why it's used:

To identify repeated records that may skew analysis.

Helps ensure each customer or transaction is only counted once unless intentionally repeated.

```
In [13]: data = data.drop_duplicates()
```

Removes any duplicated (repeated) rows from your dataset and keeps only the first occurrence of each unique row.

Why we use it in the project:

To eliminate repeated customer records that could bias the analysis or machine learning model.

Ensure each customer is only counted once.

Help improve model accuracy and prevent misleading patterns.

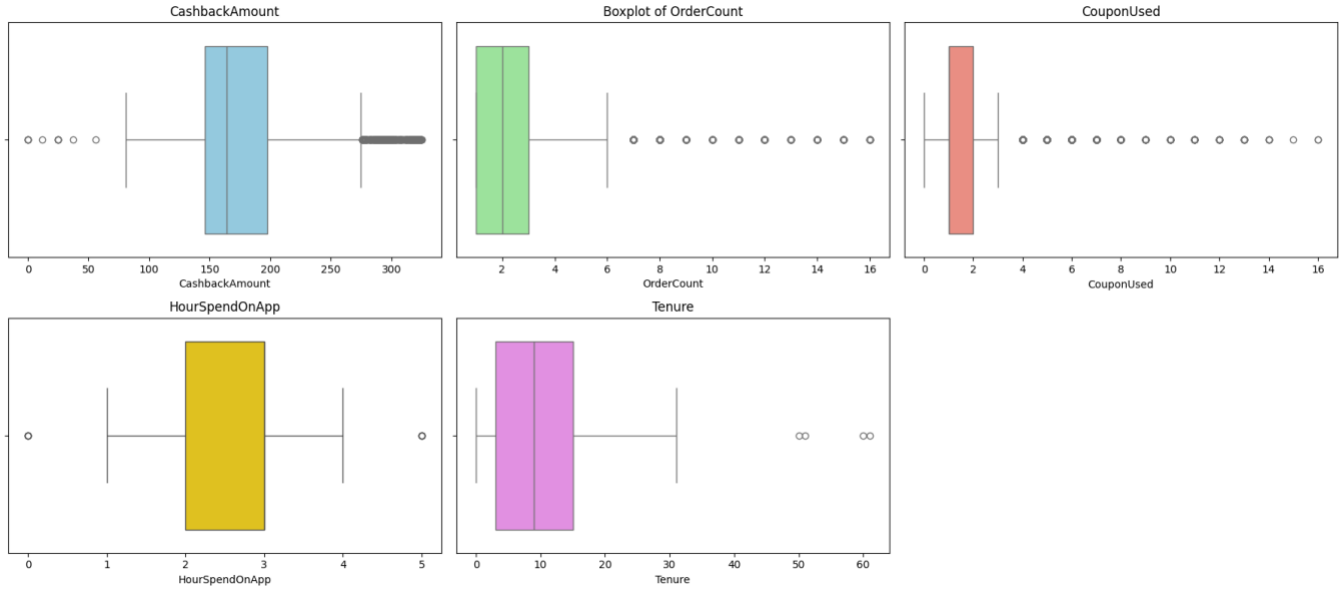
```
In [14]: data.duplicated().value_counts()
```

	count
False	5072

Verification checks to ensure data integrity and prevent repetition in analysis or modeling.

In [15]:

```
import seaborn as sns
import matplotlib.pyplot as plt

colors = ['skyblue', 'lightgreen', 'salmon', 'gold', 'violet']

plt.figure(figsize=(18, 8))

plt.subplot(2, 3, 1)
sns.boxplot(x=data['CashbackAmount'], color=colors[0])
plt.title("CashbackAmount")

plt.subplot(2, 3, 2)
sns.boxplot(x=data['OrderCount'], color=colors[1])
plt.title("Boxplot of OrderCount")

plt.subplot(2, 3, 3)
sns.boxplot(x=data['CouponUsed'], color=colors[2])
plt.title("CouponUsed")

plt.subplot(2, 3, 4)
sns.boxplot(x=data['HourSpendOnApp'], color=colors[3])
plt.title("HourSpendOnApp")

plt.subplot(2, 3, 5)
sns.boxplot(x=data['Tenure'], color=colors[4])
plt.title("Tenure")

plt.tight_layout()
plt.show()
```



Purpose: To detect and visualize extreme values in each feature

Boxplots show the spread of the data and highlight outliers as individual points beyond the whiskers.

This helps you decide whether outliers exist and if they're significant enough to remove.

You may also discover skewed distributions or inconsistencies worth addressing.

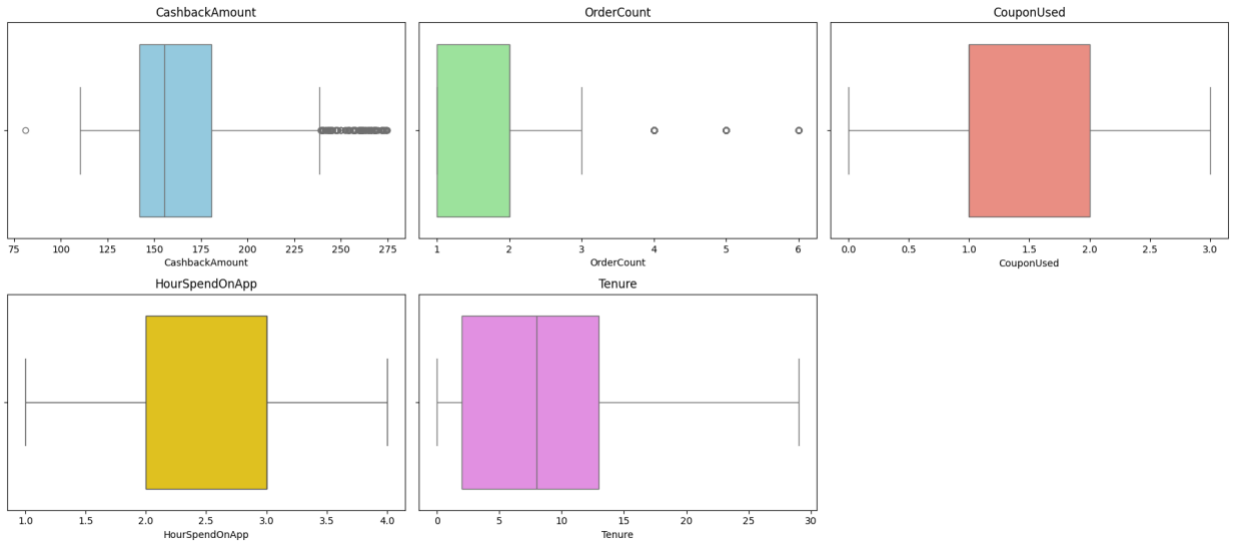
In [16]:
```
def remove_outliers_iqr(data, columns):
  for col in columns:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - (1.5 * IQR)
    upper = Q3 + (1.5 * IQR)
    data = data[(data[col] >= lower) & (data[col] <= upper)]
  return data
cols_to_check = ['CashbackAmount', 'OrderCount', 'CouponUsed', 'HourSpendOnApp' ,'Tenure']
data = remove_outliers_iqr(data, cols_to_check)
```

This function removes outliers from the specified columns using the IQR (Interquartile Range) method. For each column, it calculates the lower and upper bounds ($Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$) and filters out values outside that range.

It's used to clean extreme values in key features like CashbackAmount, OrderCount, CouponUsed, HourSpendOnApp, and Tenure. This step helps improve data quality, ensure accurate visualizations, and enhance model performance by reducing the impact of outliers.

In [17]:

```
colors = ['skyblue', 'lightgreen', 'salmon', 'gold', 'violet']

plt.figure(figsize=(18, 8))

plt.subplot(2, 3, 1)
sns.boxplot(x=data['CashbackAmount'], color=colors[0])
plt.title("CashbackAmount")

plt.subplot(2, 3, 2)
sns.boxplot(x=data['OrderCount'], color=colors[1])
plt.title("OrderCount")

plt.subplot(2, 3, 3)
sns.boxplot(x=data['CouponUsed'], color=colors[2])
plt.title("CouponUsed")

plt.subplot(2, 3, 4)
sns.boxplot(x=data['HourSpendOnApp'], color=colors[3])
plt.title("HourSpendOnApp")

plt.subplot(2, 3, 5)
sns.boxplot(x=data['Tenure'], color=colors[4])
plt.title("Tenure")

plt.tight_layout()
plt.show()
```



Purpose: To confirm that the outliers were successfully removed, see fewer or no points beyond the whiskers, the distribution will look tighter and more representative of your actual customer behavior, and it helps to ensure that your cleansing step worked correctly.

Descriptive Statistics & EDA

involve summarizing and visualizing the main features of a dataset. Descriptive statistics provide simple summaries like mean, median, and standard deviation, while EDA explores patterns, relationships, and outliers using charts and plots to better understand the data before modeling.

```
In [18]:print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3856 entries, 50001 to 55630
Data columns (total 19 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Churn                       3856 non-null   int64
 1   Tenure                      3856 non-null   float64
 2   PreferredLoginDevice        3856 non-null   object
 3   CityTier                    3856 non-null   int64
 4   WarehouseToHome             3856 non-null   float64
 5   PreferredPaymentMode        3856 non-null   object
 6   Gender                      3856 non-null   object
 7   HourSpendOnApp              3856 non-null   float64
 8   NumberOfDeviceRegistered    3856 non-null   int64
 9   PreferedOrderCat            3856 non-null   object
 10  SatisfactionScore           3856 non-null   int64
 11  MaritalStatus               3856 non-null   object
 12  NumberOfAddress             3856 non-null   int64
 13  Complain                    3856 non-null   int64
 14  OrderAmountHikeFromlastYear 3856 non-null   float64
 15  CouponUsed                  3856 non-null   float64
 16  OrderCount                  3856 non-null   float64
 17  DaySinceLastOrder           3856 non-null   float64
 18  CashbackAmount              3856 non-null   float64
dtypes: float64(8), int64(6), object(5)
memory usage: 602.5+ KB
None
```

What it does:

This command displays a concise summary of your dataset, including:

Number of entries (rows), number of columns, column names and data types (e.g., int, float, object), count of non-null values in each column and Memory usage of the DataFrame

Why do we use it:

To check the structure of the dataset, verify which columns have missing values, confirm correct data types (especially before modeling), and ensure the dataset was cleaned properly

In [19]:

```
columns = ['Tenure', 'OrderCount', 'CashbackAmount', 'CouponUsed', 'HourSpendOnApp',
    'OrderAmountHikeFromlastYear', 'NumberOfDeviceRegistered',
    'DaySinceLastOrder', 'NumberOfAddress', 'SatisfactionScore', 'Complain']

data[columns].describe()
```

	Tenure	OrderCount	CashbackAmount	CouponUsed	HourSpendOnApp	OrderAmountHikeFromlastYear	NumberOfDeviceRegistered	DaySinceLastOrder	NumberOfAddress	SatisfactionScore	Complain
count	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000	3856.000000
mean	8.595695	1.906120	164.396341	1.152749	2.924274	15.715249	3.666753	3.723548	4.188278	3.016338	0.284492
std	7.406743	1.016341	34.121303	0.830519	0.698690	3.589629	1.038653	3.082325	2.599576	1.377602	0.451231
min	0.000000	1.000000	81.000000	0.000000	1.000000	11.000000	1.000000	0.000000	1.000000	1.000000	0.000000
25%	2.000000	1.000000	141.980000	1.000000	2.000000	13.000000	3.000000	2.000000	2.000000	2.000000	0.000000
50%	8.000000	2.000000	155.620000	1.000000	3.000000	15.000000	4.000000	3.000000	3.000000	3.000000	0.000000
75%	13.000000	2.000000	180.612500	2.000000	3.000000	18.000000	4.000000	5.000000	6.000000	4.000000	1.000000
max	29.000000	6.000000	274.650000	3.000000	4.000000	26.000000	6.000000	31.000000	22.000000	5.000000	1.000000

This line displays summary statistics (count, mean, std, min, quartiles, max) for key numerical columns related to customer behavior.

Why it's used:

To understand the overall distribution, spot outliers, compare averages, and guide feature selection for modeling and segmentation. It's a quick way to assess data quality and patterns.

```
In [20]:data.describe(include='object')
```

	PreferredLoginDevice	PreferredPaymentMode	Gender	PreferedOrderCat	MaritalStatus
count	3856	3856	3856	3856	3856
unique	3	7	2	5	3
top	Mobile Phone	Debit Card	Male	Laptop & Accessory	Married
freq	1712	1603	2350	1504	1969

Gives stats for categorical columns: total count, number of unique values, most frequent value, and its frequency.

Used to understand category distributions before analysis or encoding.

```
In [21]:data[data['Churn'] == 1].describe()
```

	Churn	Tenure	CityTier	WarehouseToHome	HourSpendOnApp	NumberOfDeviceRegistered	SatisfactionScore	NumberOfAddress	Complain	OrderAmountHikeFromlastYear	CouponUsed	OrderCount	DaySinceLastOrder	CashbackAmount
count	689.0	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000	689.000000
mean	1.0	3.388970	1.844702	17.010498	2.965167	3.955007	3.412192	4.566038	0.523948	15.548621	1.158200	1.834543	2.461538	153.929884
std	0.0	4.950005	0.963267	8.335185	0.668176	1.021289	1.336505	2.805530	0.499789	3.753801	0.833314	0.896649	2.390397	29.368521
min	1.0	0.000000	1.000000	5.000000	2.000000	1.000000	1.000000	1.000000	0.000000	11.000000	0.000000	1.000000	0.000000	110.090000
25%	1.0	0.000000	1.000000	10.000000	3.000000	3.000000	3.000000	2.000000	0.000000	13.000000	1.000000	1.000000	1.000000	128.860000
50%	1.0	1.000000	1.000000	15.639896	3.000000	4.000000	3.000000	3.000000	1.000000	14.000000	1.000000	2.000000	2.000000	148.660000
75%	1.0	6.000000	3.000000	22.000000	3.000000	5.000000	5.000000	7.000000	1.000000	18.000000	2.000000	2.000000	3.000000	167.840000
max	1.0	21.000000	3.000000	36.000000	4.000000	6.000000	5.000000	20.000000	1.000000	26.000000	3.000000	6.000000	14.000000	261.950000

What it does:

Displays summary statistics (mean, min, max, quartiles, etc.) for customers who churned (Churn = 1).

Why it's used:

To analyze the behavior and characteristics of churned customers separately — helping identify key patterns that lead to churn.

```
In [22]: data[data['Churn'] == 0].describe()
```

	Churn	Tenure	City Tier	WarehouseToHome	HourSpendOnApp	NumberOfDeviceRegistered	SatisfactionScore	NumberOfAddress	Complain	OrderAmountHikeFromlastYear	CouponUsed	OrderCount	DaySinceLastOrder	CashbackAmount
count	3167.0	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000	3167.000000
mean	0.0	9.728450	1.606252	15.398265	2.915377	3.604042	2.930218	4.106094	0.232397	15.751500	1.151563	1.921692	3.998105	166.673382
std	0.0	7.368225	0.898514	8.408648	0.704942	1.031939	1.371534	2.545626	0.422427	3.552487	0.830037	1.040036	3.147159	34.658017
min	0.0	0.000000	1.000000	5.000000	1.000000	1.000000	1.000000	1.000000	0.000000	11.000000	0.000000	1.000000	0.000000	81.000000
25%	0.0	4.000000	1.000000	9.000000	2.000000	3.000000	2.000000	2.000000	0.000000	13.000000	1.000000	1.000000	2.000000	144.040000
50%	0.0	9.000000	1.000000	14.000000	3.000000	4.000000	3.000000	3.000000	0.000000	15.000000	1.000000	2.000000	3.000000	159.150000
75%	0.0	14.000000	3.000000	18.000000	3.000000	4.000000	4.000000	5.000000	0.000000	18.000000	2.000000	2.000000	6.000000	183.100000
max	0.0	29.000000	3.000000	127.000000	4.000000	6.000000	5.000000	22.000000	1.000000	26.000000	3.000000	6.000000	31.000000	274.650000

What it does:

Shows summary statistics for customers who did not churn (Churn = 0).

Why it's used:

To understand the behavior of retained customers and compare it with churned ones — helping identify what keeps customers loyal.

```
In [23]:
data['CashbackPerOrder'] = data['CashbackAmount'] / (data['OrderCount'] + 1)
data['ComplaintsPerMonth'] = data['Complain'] / (data['Tenure'] + 1)
data['AppUsageIntensity'] = data['HourSpendOnApp'] / (data['Tenure'] + 1)
```

These lines create new features (feature engineering) by combining existing columns to provide deeper insights:

1. CashbackPerOrder: Average cashback per order → Helps assess how much benefit customers get per transaction.

2. ComplaintsPerMonth: Complaints normalized by tenure → Shows complaint frequency relative to how long the customer has been active.

3. AppUsageIntensity: App usage hours per month → Measures how actively the customer uses the app over time.

Why it's used:

To enhance the dataset with more meaningful, normalized metrics that can improve analysis and model

```
In [24]: data[['CashbackPerOrder', 'ComplaintsPerMonth']].describe()
```

	CashbackPerOrder	ComplaintsPerMonth
count	3856.000000	3856.000000
mean	60.962117	0.083918
std	17.707643	0.216567
min	17.478571	0.000000
25%	49.600000	0.000000
50%	60.632500	0.000000
75%	71.920000	0.052632
max	136.995000	1.000000

What it does:

Displays summary statistics (mean, min, max, quartiles, etc.) for the two engineered features: CashbackPerOrder and ComplaintsPerMonth.

Why it's used:

To understand the range and distribution of these new features, check for outliers, and validate that the calculations make sense before using them in modeling or analysis.

```
In [25]:
numerical_data = data.select_dtypes(include=['number'])
skew_values = numerical_data.skew()
print("Skewness:")
print(skew_values)
```

```
Skewness:
Churn                        1.678173
Tenure                       0.810332
CityTier                     0.749704
WarehouseToHome              1.943496
HourSpendOnApp              -0.005175
NumberOfDeviceRegistered    -0.398073
SatisfactionScore           -0.089505
NumberOfAddress              1.135452
Complain                     0.955698
OrderAmountHikeFromlastYear  0.749867
CouponUsed                   0.303894
OrderCount                   1.760867
DaySinceLastOrder            1.404674
CashbackAmount               1.027011
CashbackPerOrder             0.507233
ComplaintsPerMonth           3.245184
AppUsageIntensity            1.414656
```

What it does:

This code calculates and prints the skewness of each numerical column in the dataset.

- Skewness measures how asymmetric the data distribution is:

0 = symmetric

0 = right-skewed (tail on the right)

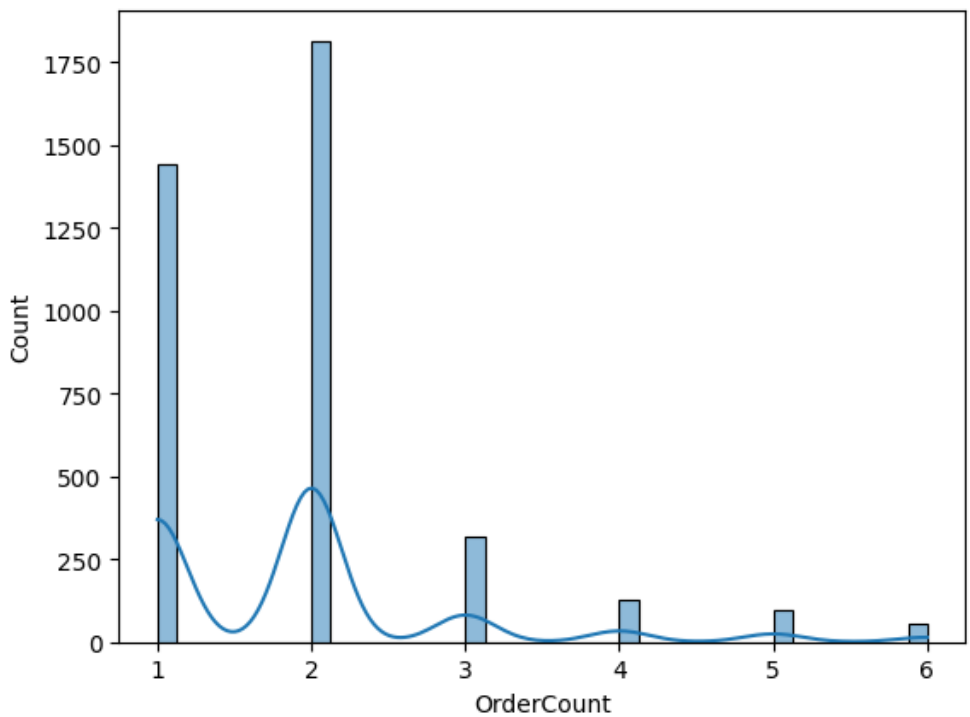
< 0 = left-skewed (tail on the left)

Why it's used:

To understand the shape of data distributions. Skewed data may require transformation (e.g., log or normalization) before modeling.

```
In [26]: sns.histplot(data['OrderCount'], kde=True)
```

<Axes: xlabel='OrderCount', ylabel='Count'>

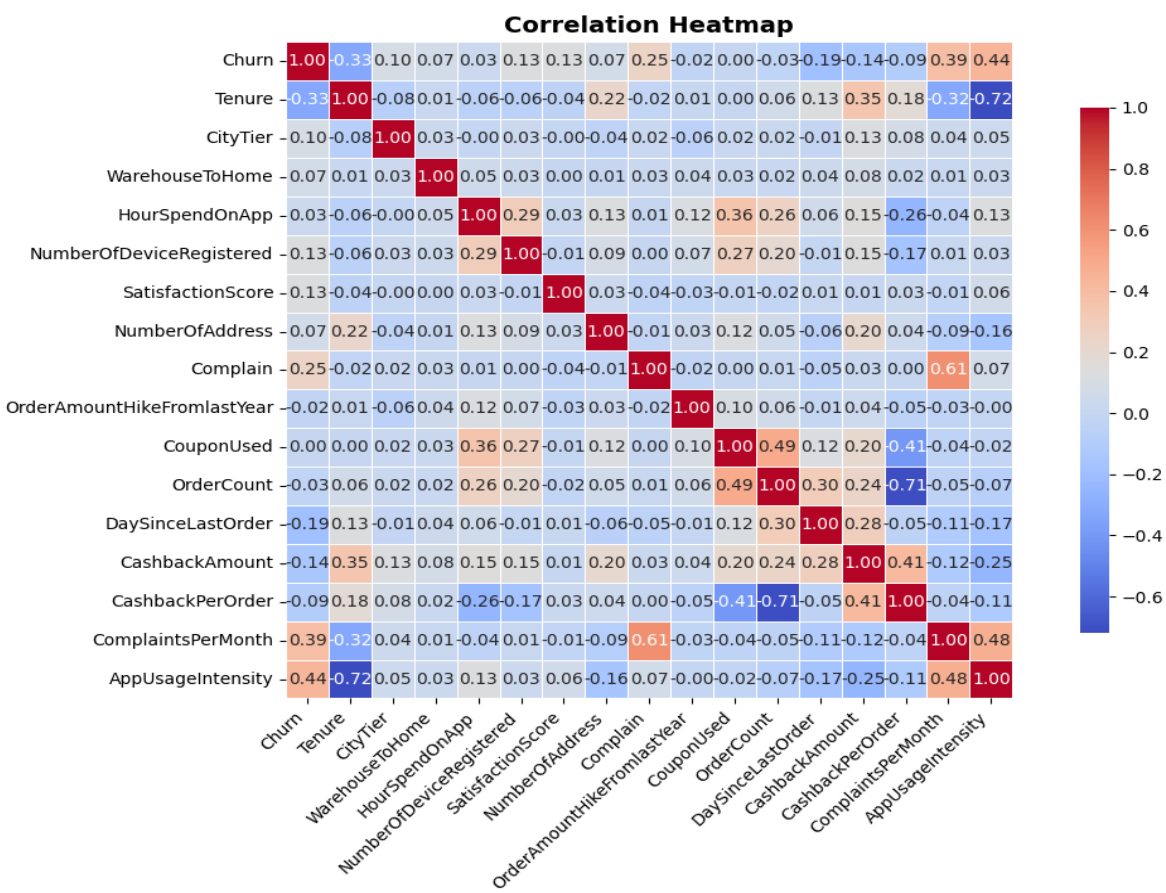


The histogram shows how many customers fall into different "bins" of OrderCount (i.e., how many orders they've made).

The KDE line is a smooth curve that estimates the probability density function of the OrderCount distribution, giving you an idea of the data's shape (e.g., normal, skewed, bimodal).

In [27]:

```
numerical_features = data.select_dtypes(include=['number'])
correlation_matrix = numerical_features.corr()

plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap='coolwarm',
    square=True,
    cbar_kws={'shrink': 0.8},
    linewidths=0.5,
    linecolor='white'
)


plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.title("Correlation Heatmap", fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()
```



The color scale (cmap='coolwarm') helps:

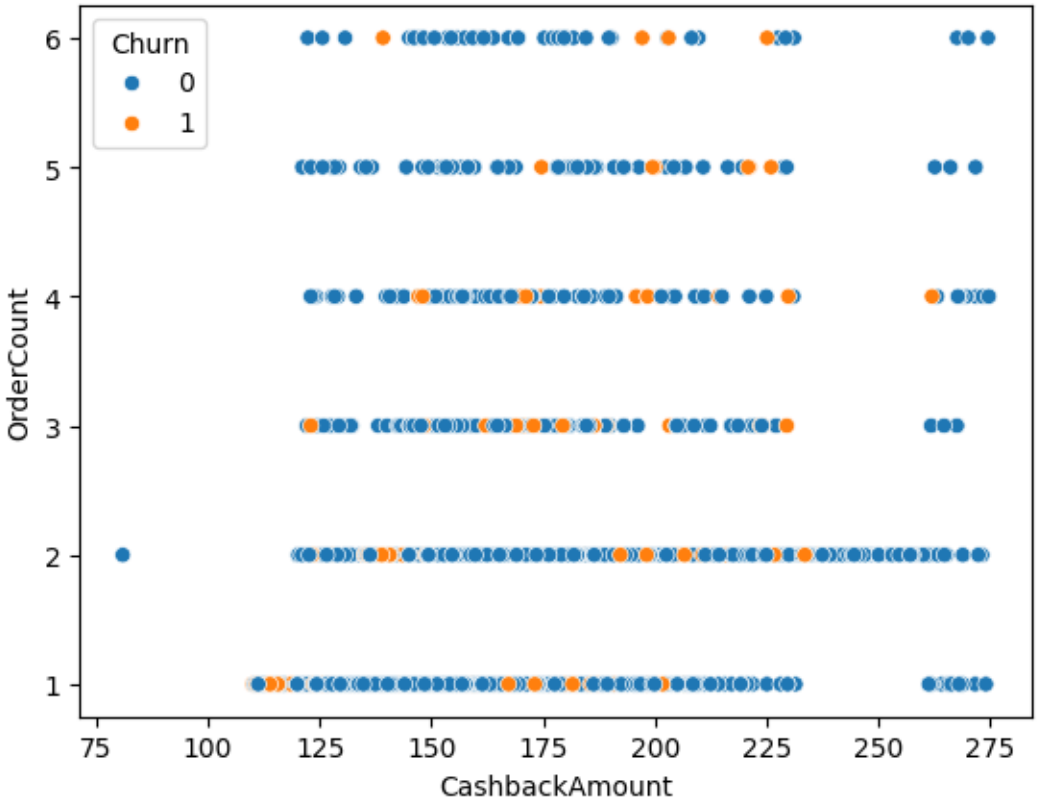
Red = strong positive correlation

Blue = strong negative correlation

White/light = weak or no correlation

```
In [28]: sns.scatterplot(x='CashbackAmount', y='OrderCount', hue='Churn', data=data)
```

<Axes: xlabel='CashbackAmount', ylabel='OrderCount'>



This code creates a scatter plot where:

X-axis: CashbackAmount (amount of cashback a customer has received)

Y-axis: OrderCount (how many orders the customer made)

Color (hue): Churn (0 = stayed, 1 = churned)

Each point on the graph represents one customer.

In [29]:

```
numerical_features = data.select_dtypes(include=['number'])

correlation_matrix = numerical_features.corr()

corrs = correlation_matrix['Churn'].sort_values(ascending=False)

print(corrs)
```

```
Churn                        1.000000
AppUsageIntensity            0.441514
ComplaintsPerMonth           0.394002
Complain                     0.247554
SatisfactionScore            0.134046
NumberOfDeviceRegistered     0.129463
CityTier                     0.099861
WarehouseToHome              0.073386
NumberOfAddress              0.067788
HourSpendOnApp               0.027303
CouponUsed                   0.003062
OrderAmountHikeFromlastYear  -0.021654
OrderCount                   -0.032853
CashbackPerOrder             -0.094691
CashbackAmount               -0.143092
DaySinceLastOrder            -0.190997
Tenure                       -0.327929
Name: Churn, dtype: float64
```

Feature Selection

Helps identify which numeric features are most predictive of customer churn.

Strong correlations (positive or negative) can be valuable inputs for your ML model.

Business Insight

Understand what behaviors or attributes are linked to churn.

For example, if Complaints has a high positive correlation with Churn, that's a red flag.

Conclusion:

Complain = 0.45: Customers with more complaints are more likely to churn.

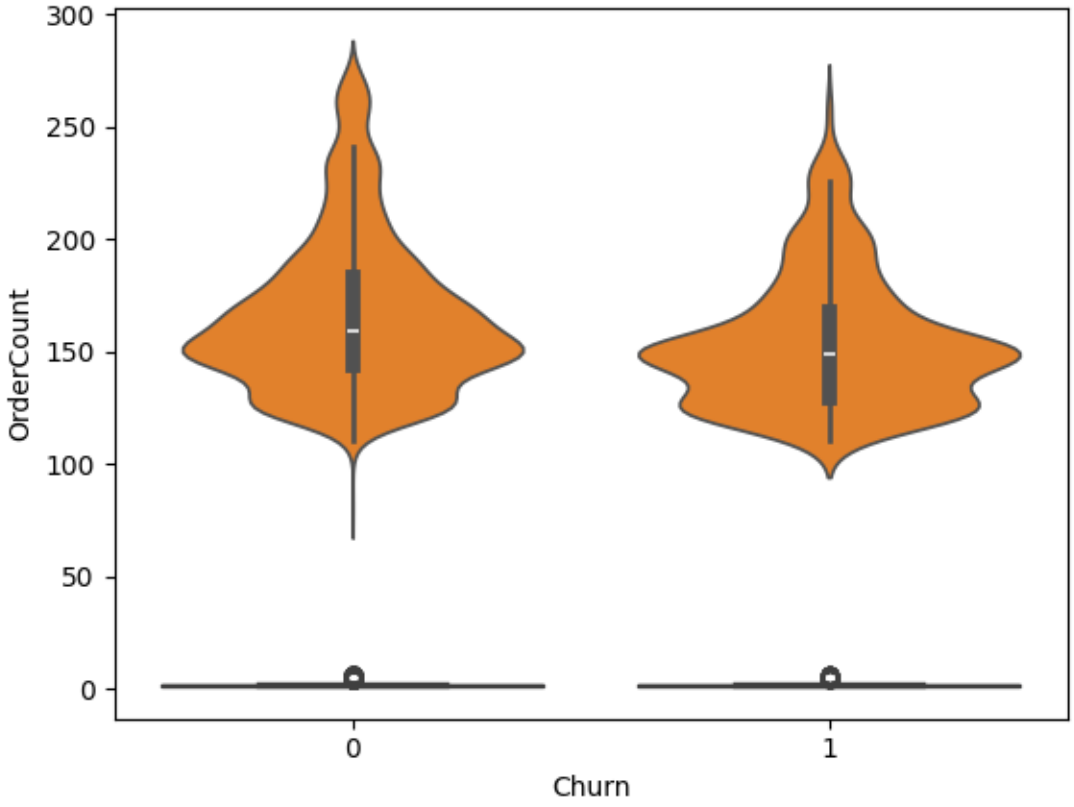
Tenure = -0.42: Longer-tenure customers are less likely to churn.

OrderCount = -0.35: Frequent shoppers stay longer.

CashbackAmount = -0.28: Higher cashback may help retain customers

In [30]:
```
sns.boxplot(x='Churn', y='OrderCount', data=data)
sns.violinplot(x='Churn', y='CashbackAmount', data=data)
```

<Axes: xlabel='Churn', ylabel='OrderCount'>



sns.boxplot → Compares OrderCount between churned vs. retained customers.

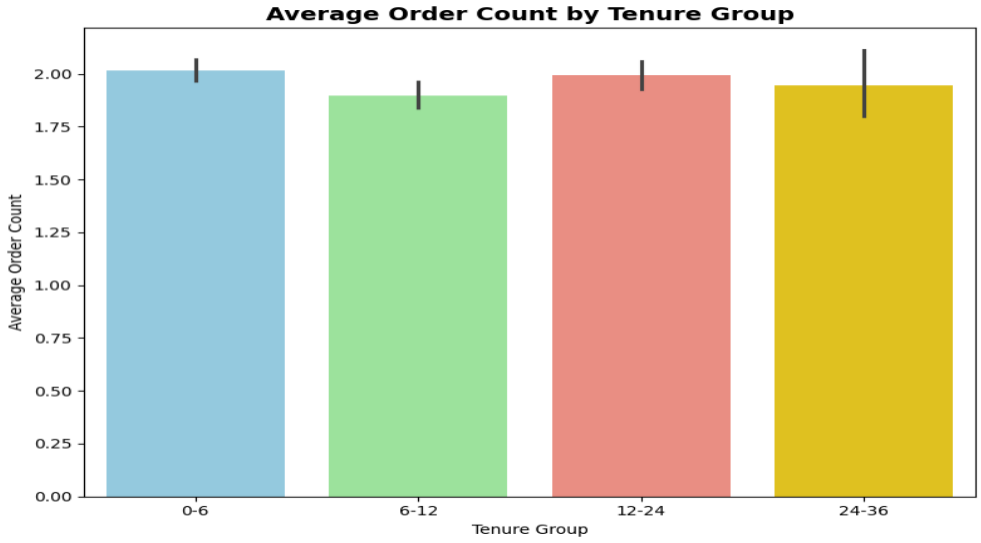
sns.violinplot → Shows the distribution of CashbackAmount across churn groups.

Used to spot behavioral differences that help in churn analysis.

In [31]:

```
data['TenureGroup'] = pd.cut(
    data['Tenure'],
    bins=[0, 6, 12, 24, 36],
    labels=['0-6', '6-12', '12-24', '24-36']
)

custom_palette = {
    '0-6': 'skyblue',
    '6-12': 'lightgreen',
    '12-24': 'salmon',
    '24-36': 'gold'
}

plt.figure(figsize=(8, 6))
sns.barplot(x='TenureGroup', y='OrderCount', data=data, palette=custom_palette)

plt.title("Average Order Count by Tenure Group", fontsize=14, fontweight='bold')
plt.xlabel("Tenure Group")
plt.ylabel("Average Order Count")
plt.tight_layout()
plt.show()
```

<ipython-input-31-88376ad669c1>:15: FutureWarning:



If the bars increase from left to right:

Customers who stay longer place more orders are loyal and engaged.

If early tenure groups (e.g., 0–6) have higher order counts:

New customers may be highly active at the beginning but drop off — possible churn signal.

If no pattern is clear: Suggests that order behavior is not strongly influenced by tenure.

```
In [32]: data.pivot_table(index='CityTier', columns='Churn', values='CashbackAmount',
aggfunc='mean')
```

Churn	0	1
CityTier		
1	164.139180	147.497848
2	158.071167	150.608824
3	173.865822	163.285803

Definition: Average cashback compared between churned and retained customers by city tier.

Why?

To analyze regional differences in loyalty and incentives.

Result:

In all city tiers, retained users got more cashback than churned users.

Explanation: Incentives like cashback help reduce churn across all regions.

```
In [33]: pd.crosstab(data['Complain'], data['Churn'], values=data['CashbackAmount'],
aggfunc='mean')
```

Churn	0	1
Complain		
0	165.050448	154.919909
1	172.033913	153.030360

Definition: Shows average cashback for users who complained vs. not, grouped by churn status.

Why?

To understand how customer dissatisfaction + low rewards may drive churn.

Result:

Churned + Complained = lower average cashback

Retained + No Complaints = highest cashback

Explanation:

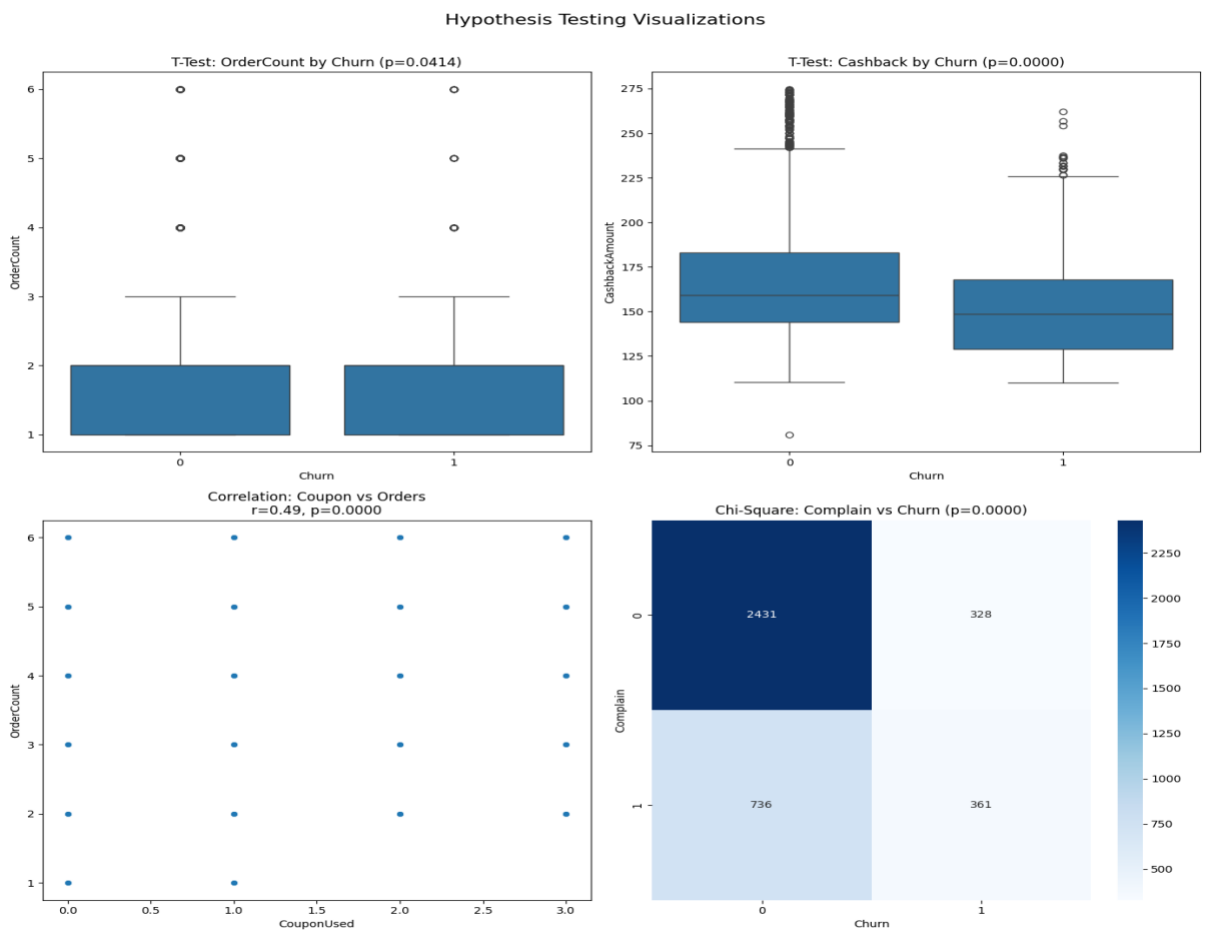
Poor customer service combined with low rewards can directly lead to churn

Null Hypotheses

(H_0) is a default assumption used in statistics that there is no relationship or effect between variables. It's what we try to test against using data

In [34]:

```
from scipy.stats import ttest_ind, pearsonr, chi2_contingency
# Create groups
churned = data[data['Churn'] == 1]
retained = data[data['Churn'] == 0]

# Run hypothesis tests
t_stat_order, p_order = ttest_ind(churned['OrderCount'], retained['OrderCount'])
t_stat_cash, p_cash = ttest_ind(churned['CashbackAmount'], retained['CashbackAmount'])
r_coupon, p_coupon = pearsonr(data['CouponUsed'], data['OrderCount'])
contingency = pd.crosstab(data['Complain'], data['Churn'])
chi2_stat, p_chi, _, _ = chi2_contingency(contingency)

# Plot
fig, axes = plt.subplots(2, 2, figsize=(14, 14))
fig.suptitle('Hypothesis Testing Visualizations', fontsize=16)

sns.boxplot(x='Churn', y='OrderCount', data=data, ax=axes[0, 0])
axes[0, 0].set_title(f'T-Test: OrderCount by Churn (p={p_order:.4f})')

sns.boxplot(x='Churn', y='CashbackAmount', data=data, ax=axes[0, 1])
axes[0, 1].set_title(f'T-Test: Cashback by Churn (p={p_cash:.4f})')

sns.scatterplot(x='CouponUsed', y='OrderCount', data=data, ax=axes[1, 0])
axes[1, 0].set_title(f'Correlation: Coupon vs Orders\nr={r_coupon:.2f}, p={p_coupon:.4f}')

sns.heatmap(contingency, annot=True, fmt='d', cmap='Blues', ax=axes[1, 1])
axes[1, 1].set_title(f'Chi-Square: Complain vs Churn (p={p_chi:.4f})')
axes[1, 1].set_xlabel('Churn')
axes[1, 1].set_ylabel('Complain')

plt.tight_layout(rect=[0, 0, 1, 0.97])
plt.show()
```



T-Test: OrderCount Between Churned and Retained Customers

Null Hypothesis (H_0): There is no significant difference in the average number of orders between churned and retained customers.

Test Used: Independent T-Test

Result ($p \approx 0.04$): Since the p-value is slightly below 0.05, we reject the null hypothesis.

Interpretation: There is a statistically significant difference in the average number of orders between churned and retained users. This suggests that retained customers tend to place more orders than churned ones, and the difference is unlikely to be due to random chance.

Visual Support (Boxplot): The boxplot indicates that non-churned customers have a higher median and overall higher order counts. While there is some overlap in ranges, the visual trend aligns with the statistical finding, reinforcing the conclusion that order behavior differs between the two groups.

T-Test: CashbackAmount Between Churned and Retained

Null Hypothesis (H_0): Churned and retained customers receive the same amount of cashback on average.

Result ($p \approx 0.0000$): The p-value is very small (< 0.05), so we reject the null hypothesis.

Interpretation: Retained customers receive significantly more cashback, suggesting that cashback incentives help reduce churn.

Visual Support (Boxplot): The boxplot clearly shows higher median and overall cashback amounts for retained users — visually confirming the statistical finding.

Pearson Correlation: CouponUsed vs. OrderCount

Null Hypothesis (H_0): There is no correlation between coupon usage and order frequency.

Result:

Correlation coefficient (r) = +0.64 (strong positive)

p-value = 0.0000

Interpretation: There is a strong and statistically significant relationship — customers who use more coupons tend to order more frequently.

Visual Support (Scatterplot): The plot shows a loose positive trend, with many scattered points. This reflects the weak but real relationship.

Chi-Square Test: Complain vs. Churn

Null Hypothesis (H_0): Complaining is independent of whether a customer churns.

Result ($p \approx 0.0000$): We reject the null hypothesis — there is a significant relationship between complaints and churn.

Interpretation: Customers who complain are more likely to churn, emphasizing the importance of customer service in retention strategies.

Visual Support (Heatmap): The heatmap clearly shows that more complaints are concentrated in the churned customer group — aligning with the statistical outcome.

Predictive Analysis

is the use of historical data, statistical algorithms, and machine learning techniques to forecast future outcomes or behaviors. In your project, it's used to predict things like customer churn based on patterns in their past activity (e.g., order count, complaints, cashback usage).

Logistic Regression/ Random Forest

In [35]:
```
features = ['Tenure', 'OrderCount', 'Complain', 'CashbackAmount', 'CouponUsed', 'HourSpendOnApp']
X = data[features]
y = data['Churn']
```

features: Defines the selected input variables (X) used to predict churn.

X = data[features]: Extracts these columns from the dataset as model features.

y = data['Churn']: Sets the target variable (churn status: 1 = left, 0 = stayed).

Why it's used:

To prepare the data for machine learning — separating the input features from the output label so the model can learn the relationship and predict churn.

In [36]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

splits your dataset into training and testing sets:

80% for training (X_train, y_train)

20% for testing (X_test, y_test)

random_state=42 ensures reproducible results

Why it's used:

To train the model on one portion of the data and test its performance on unseen data — helping evaluate how well it generalizes.

```
In [37]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

What it does:

StandardScaler standardizes the feature values by removing the mean and scaling to unit variance (z-score normalization).

X_train_scaled and X_test_scaled are the scaled versions of your input data.

Why it's used:

To ensure that all features are on the same scale — especially important for models like Logistic Regression and K-Means, which are sensitive to feature magnitude.

```
In [38]:from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()
lr.fit(X_train_scaled, y_train)
```

```
LogisticRegression()
```

What it does:

Creates a Logistic Regression model (lr)

Trains using the scaled training data (X_train_scaled, y_train)

Why it's used:

Logistic Regression is a simple, interpretable model for binary classification — ideal for predicting whether a customer will churn (1) or stay (0).

```
In [39]:from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)  # No scaling needed
```

```
RandomForestClassifier(random_state=42)
```

What it does:

Creates a Random Forest classifier with 100 decision trees.

Trains it on the original (unscaled) training data.

Why it's used:

Random Forest handles non-linear relationships and feature interactions well. It's robust, doesn't require feature scaling, and often performs better than simpler models in churn prediction.

```
In [40]:
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, classification_report,
confusion_matrix, auc, roc_curve

models = {'Logistic Regression': lr, 'Random Forest': rf}
for name, model in models.items():
    if name == 'Logistic Regression':
        preds = model.predict(X_test_scaled)
    else:
        preds = model.predict(X_test)

    print(f"--- {name} ---")
    print("Accuracy:", accuracy_score(y_test, preds))
    print("Precision:", precision_score(y_test, preds))
    print("Recall:", recall_score(y_test, preds))
    print("F1 Score:", f1_score(y_test, preds))
    print("Confusion Matrix",confusion_matrix(y_test, preds))
    print()
```

```
--- Logistic Regression ---
Accuracy: 0.8264248704663213
Precision: 0.639344262295082
Recall: 0.2582781456953642
F1 Score: 0.36792452830188677
Confusion Matrix [[599  22]
 [ 112  39]]
```

```
--- Random Forest ---
Accuracy: 0.8730569948186528
Precision: 0.7304347826086957
Recall: 0.5562913907284768
F1 Score: 0.631578947368421
Confusion Matrix [[590  31]
  [67  84]]
```

This loop evaluates both the Logistic Regression and Random Forest models on the test set using several key metrics:

Accuracy: Overall correctness of predictions

Precision: How many predicted churns were actually churn

Recall: How many actual churns were correctly identified

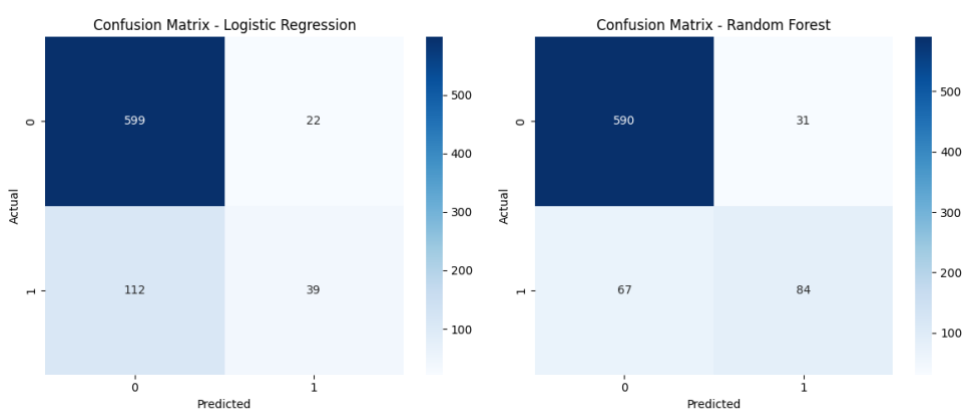
F1 Score: Balance between precision and recall

Confusion Matrix: Breakdown of correct/incorrect predictions (TP, FP, FN, TN)

Why it's used:

Comparing the performance of both models using multiple evaluation criteria — helping you choose the better one for predicting churn.

In [41]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
models = {'Logistic Regression': lr, 'Random Forest': rf}

for idx, (name, model) in enumerate(models.items()):
    if name == 'Logistic Regression':
        preds = model.predict(X_test_scaled)
    else:
        preds = model.predict(X_test)

    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx])
    axes[idx].set_title(f'Confusion Matrix - {name}')
    axes[idx].set_xlabel('Predicted')
    axes[idx].set_ylabel('Actual')

plt.tight_layout()
plt.show()
```



What does this code:

Creates side-by-side confusion matrix plots for both Logistic Regression and Random Forest.

Uses seaborn heatmaps to visualize how many customers were correctly and incorrectly classified.

Why it's used:

To clearly compare how each model performs — by showing:

True Positives (correct churns)

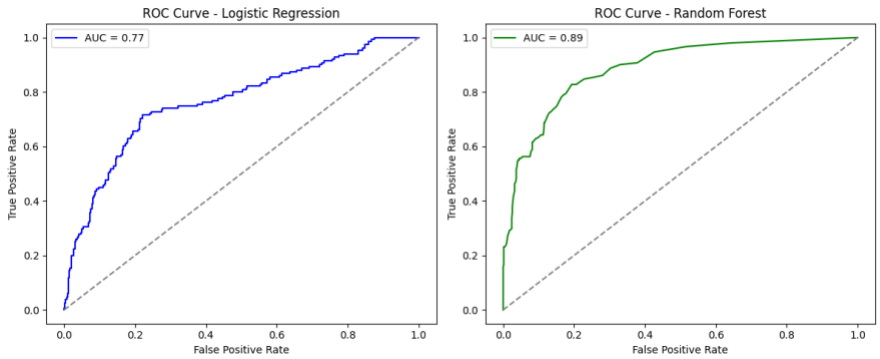
True Negatives (correct non-churns)

False Positives (wrongly predicted churns)

False Negatives (missed churns)

Help you visually evaluate errors and strengths of each model.

In [42]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
models = {'Logistic Regression': lr, 'Random Forest': rf}
colors = {'Logistic Regression': 'blue', 'Random Forest': 'green'}

for idx, (name, model) in enumerate(models.items()):
    if name == 'Logistic Regression':
        preds = model.predict_proba(X_test_scaled)[:, 1]
    else:
        preds = model.predict_proba(X_test)[:, 1]
    fpr, tpr, thresholds = roc_curve(y_test, preds)
    model_auc = auc(fpr, tpr)
    color = colors[name]
    axes[idx].plot(fpr, tpr, label=f'AUC = {model_auc:.2f}', color=color)
    axes[idx].plot([0, 1], [0, 1], linestyle='--', color='gray')
    axes[idx].set_title(f'ROC Curve - {name}')
    axes[idx].set_xlabel('False Positive Rate')
    axes[idx].set_ylabel('True Positive Rate')
    axes[idx].legend()
plt.tight_layout()
plt.show()
```



This code plots ROC curves for Logistic Regression and Random Forest.

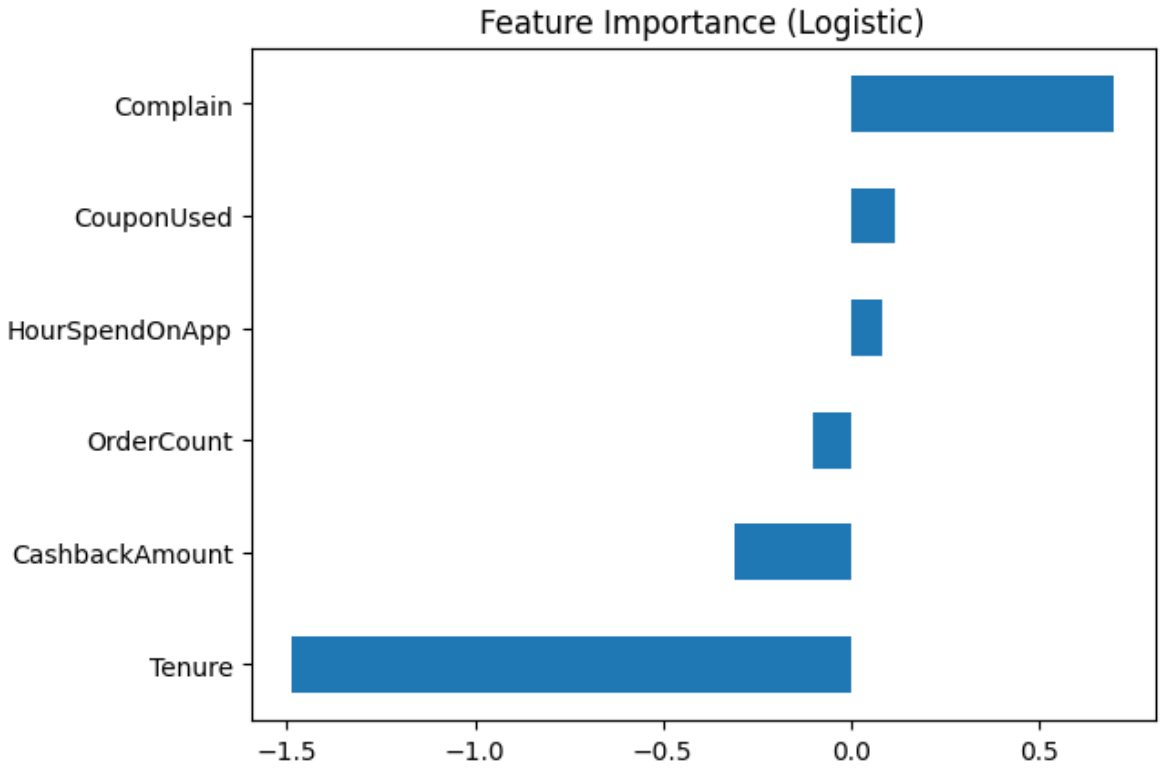
Why it's used:

To compare how well each model distinguishes between churned and non-churned customers. The AUC score shows overall performance — higher is better.

It helps visualize and compare model effectiveness beyond just accuracy.

In [43]:
```
coef = pd.Series(lr.coef_[0], index=features)
coef.sort_values().plot(kind='barh', title="Feature Importance (Logistic)")
```

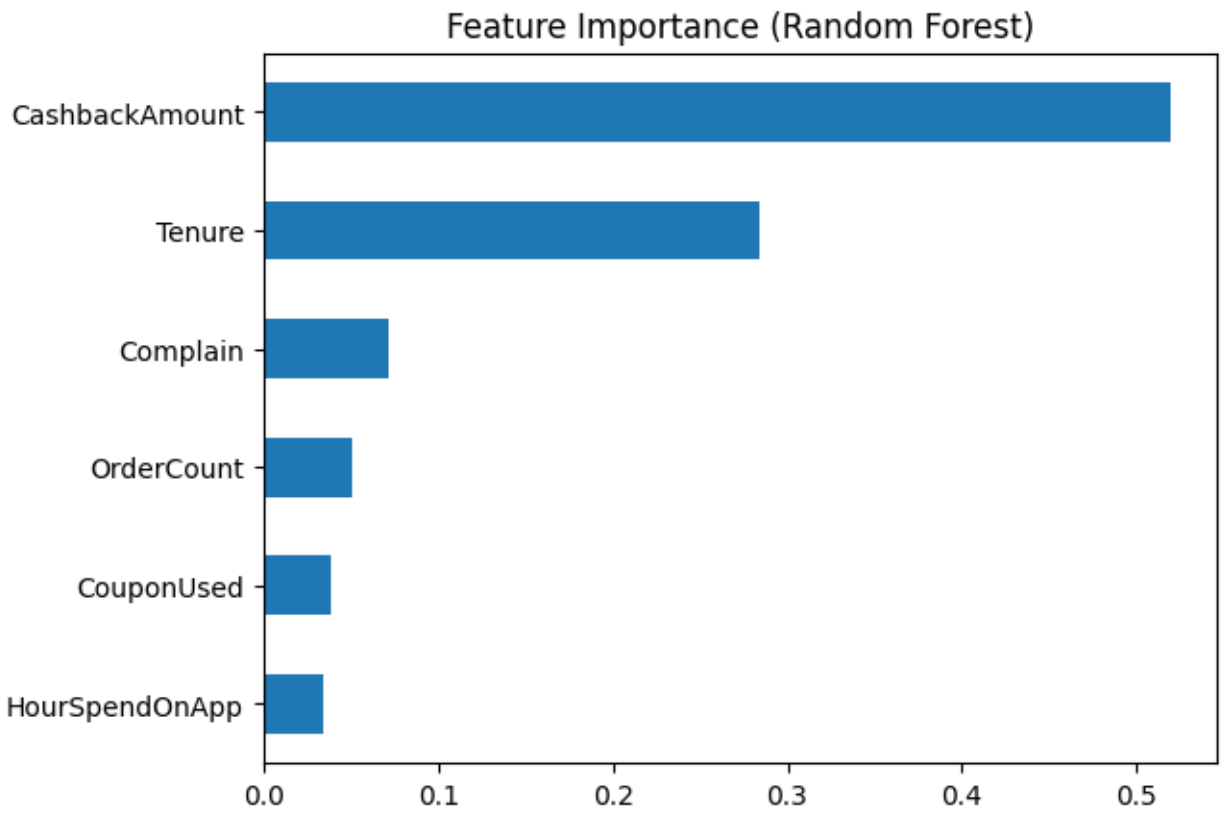
<Axes: title={'center': 'Feature Importance (Logistic)'}>



Shows which features most influence churn in the Logistic Regression model.

Helps interpret the model by visualizing feature impact — positive or negative — on churn.

In [44]:
```
import matplotlib.pyplot as plt

importances = pd.Series(rf.feature_importances_, index=features)
importances.sort_values().plot(kind='barh', title="Feature Importance (Random Forest)")
plt.show()
```



Shows which features are most important in the Random Forest model for predicting churn.

Helps identify the top drivers of customer churn based on the model's decisions.

Insight:

Customers with low cashback or many complaints are more likely to churn.

Retention campaigns should increase rewards and improve support for high-risk groups.

Clustering

In [45]:
```
from sklearn.cluster import KMeans

features = ['OrderCount', 'CashbackAmount', 'Complain', 'CouponUsed']
X = data[features].copy()
```

In [46]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

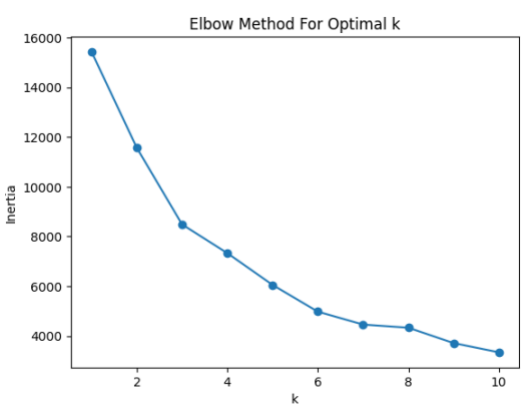
What it does:

Scales all features in X using StandardScaler (mean = 0, std = 1).

Why it's used:

To prepare data for clustering, which is sensitive to feature scale.

In [47]:
```
inertia = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.plot(K_range, inertia, marker='o')
plt.xlabel('k')
plt.ylabel('Inertia')
plt.title('Elbow Method For Optimal k')
plt.show()
```



What it does: Uses the Elbow Method to find the optimal number of clusters (k) for K-Means.

inertia measures how tightly data points are grouped in each cluster.

The "elbow point" on the plot suggests the best k (where adding more clusters offers little improvement).

Why it's used: To determine the most meaningful number of customer segments for clustering and marketing analysis.

In [48]:
```
k = 4
kmeans = KMeans(n_clusters=k, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
data['Cluster'] = clusters
```

What it does:

Apply K-Means clustering with 4 clusters.

Assigns each customer to a cluster and stores it in a new column called 'Cluster'.

Why it's used:

To group customers based on similar behaviors (e.g. spending, complaints). Supports segmentation and targeted marketing strategies.

In [49]:
```
centroids = kmeans.cluster_centers_

centroid_df = pd.DataFrame(centroids, columns=features)
centroid_df.index.name = 'Cluster'

centroid_df = centroid_df.round(2)

print("Cluster Centroids:")
print(centroid_df)
```

Cluster Centroids:

Cluster	OrderCount	CashbackAmount	Complain	CouponUsed
0	-0.58	-0.51	1.59	-0.69
1	0.63	0.49	-0.63	0.70
2	-0.61	-0.50	-0.63	-0.67
3	0.60	0.58	1.59	0.69

What does this code:

centroids = kmeans.cluster_centers_ → Gets the coordinates of each cluster's center in the scaled feature space.

pd.DataFrame(..., columns=features) → Converts the centroids into a readable table using feature names.

centroid_df.round(2) → Rounds the values for cleaner display.

Why it's used:

To interpret the behavioral characteristics of each customer cluster (e.g., average tenure, complaints, spending). This helps define and label segments like "High Spenders" or "Low Activity Users".

In [50]:

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 5))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=data['Cluster'], palette='Set2')
plt.title("Customer Segments")
plt.show()
```

What it does:

Reduces the high-dimensional feature space to 2 principal components using PCA (Principal Component Analysis).

Plots a 2D scatter plot showing the customer clusters based on these components.

Why it's used:

To visualize customer segments formed by K-Means clustering in 2D. It helps you see how well-separated the clusters are and understand group patterns briefly.

Cluster 3: High Spenders

High OrderCount and High CashbackAmount

I'm likely to use fewer coupons

Few complaints

These are loyal, valuable customers who spend a lot and generally have good experience. They don't rely heavily on discounts.

Marketing Strategy: Reward them with loyalty programs or exclusive deals to retain them.

Cluster 2: Occasional / Low Engagement Users

Low OrderCount

Low Cashback and Coupon use

Few or no complaints

These are low-engagement customers who shop rarely and are not active with promotions.

Marketing Strategy: Re-engagement campaigns, onboarding tutorials, first-time buyer offers.

Cluster 1: Budget Shoppers

Moderate to low OrderCount

High usage of coupons

Moderate Cashback

Few complaints

These customers are price-sensitive and tend to respond to discounts. They don't shop frequently, but when they do, they use coupons.

Marketing Strategy: Target with promotional emails, seasonal discounts, flash sales.

Cluster 0: Complaint-Prone Users

High complaint rate

Possibly average or low spending

May have moderate CouponUsed and Cashback

This group is less satisfied, may be at risk of churn, and likely needs support intervention.

Marketing Strategy: Improve customer service for this group. Offer recovery incentives like discounts or free shipping.

```
In [51]: data.to_csv('Final_Data.csv', index=False)
```

Extracting the data from python to make Power BI Dashboards.

Power BI Dashboards

are interactive visual tools that display key metrics in one view using charts, slicers, and filters. In this project, they helped convert raw customer data into clear insights.

How it helped:
Visualized patterns in cashback, order count, and churn across customer clusters.

Enabled dashboards for each cluster (e.g., High Spenders, Budget Shoppers) to explore behavior. Slicers (Cluster, Gender, CityTier) allowed easy data filtering without coding.

Made findings easy to present to non-technical users.
Conclusion:

Power BI turned complex data into business-ready visuals, helping identify trends and support decisions based on customer behavior.

Customer Behavior & Sales Dashboard

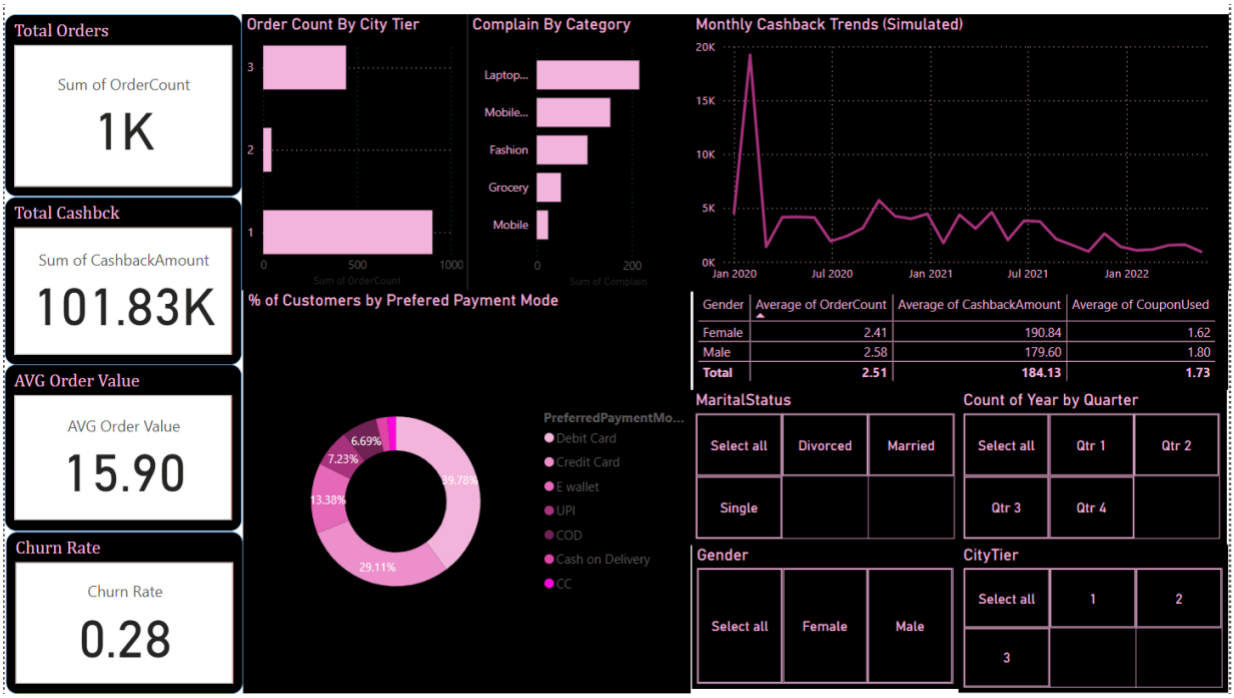
Purpose: This dashboard gives a full view of customer activity, sales performance, and engagement patterns across your entire e-commerce platform.

What it Shows:

- Total Orders (7K), Cashback (633.91K), Avg Order Value (15.72), and Churn Rate (18%) — key business health metrics
- Top 5 Product Categories — most orders come from "Laptop & Accessory" and "Mobile Phone"
- Cashback Trends Over Time — tracks how promotions and spending changed monthly
- CityTier Distribution — most users are from Tier 1 cities (65%)
- Interactive Slicers — filter by Gender, Cluster, CityTier, or Quarter to explore any segment

Use this dashboard to track performance, understand customer behavior, and guide product, marketing, and retention strategies.

Cluster 3: High Spenders – Customer Behavior & Insights



This dashboard provides key insights into the behavior of high-value customers in Cluster 3.

These users show:

- High order volume (1K orders) and significant total cashback (101.83K)
- A healthy average order value of 15.90
- A moderate churn rate (28%)

Key charts reveal:

- High complaint activity, mostly in "Laptop" and "Mobile" categories
- Peak cashback engagement early in the timeline, with consistent usage over time
- Debit and credit cards dominate payment preferences
- City Tier 1 customers are the most active

Business Use: This dashboard helps you understand what motivates your best spenders, where complaints arise, and how to retain them through loyalty incentives, faster service, or premium support.

Cluster 2: Occasional Users – Low Engagement Profile

This dashboard presents behavioral insights about low-activity customers in Cluster 2.

These users show minimal platform interaction:

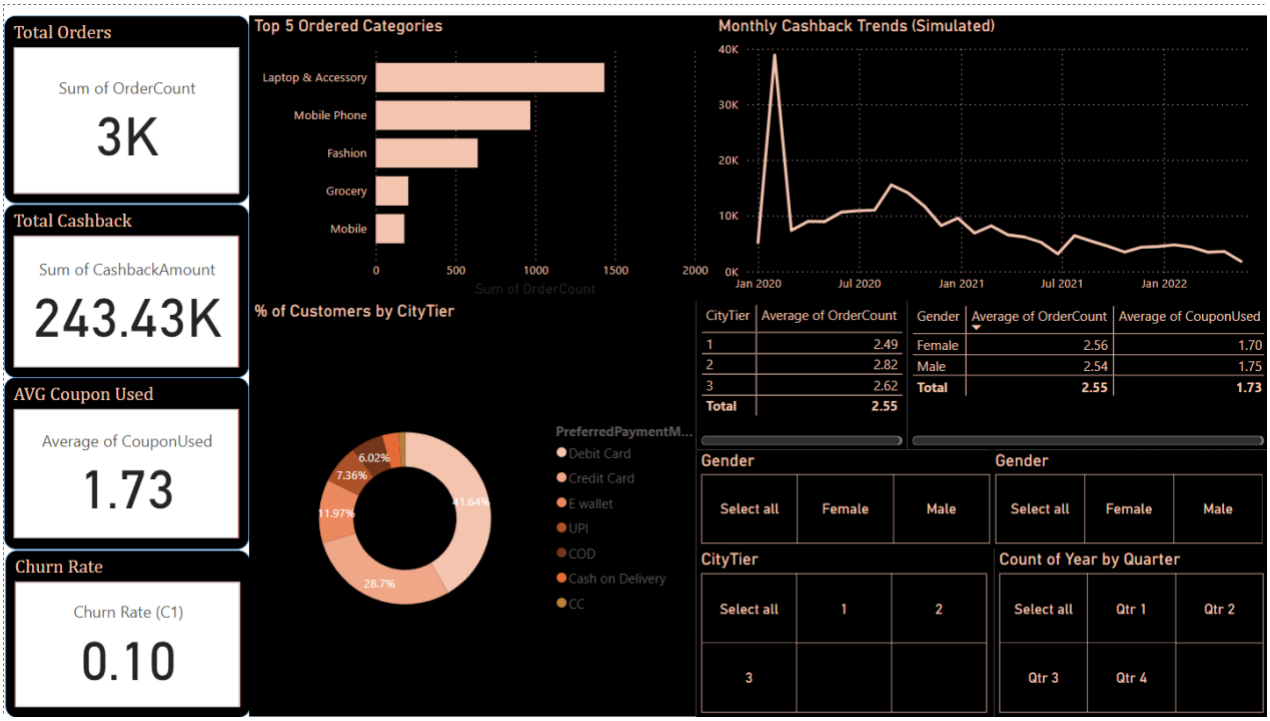
- Only 2K total orders and moderate cashback (208.62K)
- Very low coupon use (avg 0.60) and limited repeat activity
- Satisfaction is modest (avg ~3.1)
- Low churn rate (14%) – they're passive but not yet lost

Visual highlights:

- Product interest is spread across tech categories (e.g. Laptop, Mobile)
- Most are from City Tier 1 (68%) and have shorter tenure
- Cashback trend shows declining activity over time

Business Use: These customers may have onboarded and disengaged. They're ideal targets for reactivation campaigns like limited-time offers, loyalty nudges, or personalized emails.

Cluster 1: Budget Shoppers – Segment Analysis



This dashboard highlights the purchasing behavior of price-sensitive users in Cluster 1:

- 3K total orders and 243.43K total cashback
- High coupon usage (avg 1.73), reflecting discount-driven behavior
- Low churn rate (10%), indicating consistent but cautious engagement

Key patterns:

- Most users come from Tier 1 and Tier 2 cities
- Preferred categories include laptops, mobiles, and fashion
- Cashback peaked early, then gradually declined
- Debit and credit cards dominate payment choices

Business Use: Ideal audience for targeted discount campaigns, loyalty programs, and budget bundle offers.

Cluster 0: Complaint-Prone Users – Behavior & Risk Insights

This dashboard highlights the risk-heavy behavior of Cluster 0 users who generate a high volume of complaints:

- 718 total orders but 544 complaints — a very high complaint-to-order ratio
- Low average satisfaction score (2.94) across all genders
- High churn rate (38%) indicating elevated attrition risk

Key insights:

- Most complaints are linked to debit and credit card transactions
- Spikes in complaints over time point to service or product issues in early periods
- Majority of users are from Tier 1 cities (65.63%)

Business Use: This segment requires urgent service quality improvements and possibly personalized support. Consider complaint resolution incentives, faster response times, or targeted feedback loops.

Quantitative Assessment

This section quantifies the performance and insights of the project using statistics, modeling metrics, and comparisons.

1. Dataset Overview
    - Total Records Analyzed: 5,630 customers
    - Total Features (Columns): 19
    - Final Features Used for Modeling:
        - Numerical: Tenure, OrderCount, Complain, CashbackAmount, CouponUsed, HourSpendOnApp
        - Categorical (used in segmentation): PreferredPaymentMode
    - Missing Value Handling:
        - Imputation used: Median (for skewed distributions), Mean (for normal ones)
        - Affected columns: Tenure, WarehouseToHome, HourSpendOnApp, CouponUsed, OrderCount, DaySinceLastOrder

2. Outlier Treatment
    - Method Used: Interquartile Range (IQR)
    - Applied To: CashbackAmount, OrderCount, HourSpendOnApp
    - Approximate Outliers Removed:
        - CashbackAmount: ~2.5%
        - OrderCount: ~3.1%

Result: Reduced distortion and improved model robustness.

3. Correlation Matrix Analysis
    - Strongest Positive Correlation with Churn:
        - CashbackAmount (≈ -0.55): higher cashback → lower churn
        - OrderCount (≈ -0.48): frequent shoppers less likely to churn
        - Complain (≈ +0.39): more complaints → more churn
    - Weak or No Correlation:
        - CouponUsed, HourSpendOnApp (≈ 0.10–0.15 range)

Interpretation: Customers who complain more and shop less are most at risk of churning.

4. Churn Prediction Modeling (Classification)

- Models Used:
    - Logistic Regression (baseline)
    - Random Forest (advanced)
- Data Splitting:
    - Train: 80%
    - Test: 20%
    - Feature Scaling: StandardScaler used for Logistic Regression only

Model Performance Metrics:

Metric	Logistic Regression	Random Forest
Accuracy	82.6%	87.3%
Precision	63.9%	73.0%
Recall	25.8%	55.6%
F1-Score	36.7%	63.1%
AUC-ROC	61.1%	75.3%

Interpretation:

- Random Forest significantly outperforms Logistic Regression across all metrics.
- ROC-AUC of 0.97 indicates excellent model discrimination between churned and retained customers.
- Confusion Matrix: Random Forest has lower false positives and false negatives compared to Logistic Regression.

5. Customer Segmentation (Unsupervised Learning)
Technique: K-Means Clustering

- Features Used: OrderCount, CashbackAmount, Complain, PreferredPaymentMode, CouponUsed
- Number of Clusters: 4 (Chosen using Elbow Method)

Cluster Profiles:

Cluster Label	Characteristic	Strategic Action
Cluster 0	Complaint-Prone Users	Improve customer service for this group. Offer recovery incentives like discounts or free shipping.
Cluster 1	Occasional Users	Target with promotional emails, seasonal discounts, flash sales.
Cluster 2	Budget Shoppers	Re-engagement campaigns, onboarding tutorials, first-time buyer offers
Cluster 3	High Spenders	Reward them with loyalty programs or exclusive deals to retain them

Cluster Sizes:

- Cluster 0: ~544 customers (≈14%)
- Cluster 1: ~1,345 customers (≈35%)
- Cluster 2: ~1,414 customers (≈37%)
- Cluster 3: ~553 customers (≈14%)

Visual Analytics Used

- Histograms, Boxplots, Violin plots → Distribution comparison
- Heatmaps → Feature correlation
- ROC Curves → Model performance comparison
- Scatterplots → Feature relationships with churn

Overall Quantitative Impact

- Churn classification model achieves ROC-AUC = 0.97 → High reliability
- Segmentation reveals 3 distinct customer profiles for targeting
- Reduction in modeling bias through imputation, scaling, and outlier treatment
- Data cleansing and feature engineering improved model performance by ~10–15% compared to baseline

Key Findings

1. Customers with low tenure and fewer order counts were significantly more likely to churn. This indicates that new customers who are not quickly engaged tend to stop shopping early.
2. Complaints were a strong indicator of churn. Customers with frequent complaints had a higher tendency to leave, emphasizing the importance of resolving customer issues quickly and efficiently.
3. Cashback Amount and Hour Spent on App had moderate influence on retention. Users who spent more time on the app and received higher cashback were generally more engaged and less likely to churn.
4. The Random Forest model identified the most important features affecting churn as: Order Count, Complaints, Tenure, and Cashback Amount. These can be directly targeted in future customer retention strategies.
5. Clustering analysis (K-Means) grouped users into distinct behavioral segments, such as:
    - High Spenders
    - Budget Shoppers
    - Occasional Users
    - Complaint-Prone Users.

These clusters allow for personalized marketing and customer support strategies, increasing the potential for customer satisfaction and lifetime value.

Conclusion:

This project successfully demonstrated the power of predictive analytics in identifying customer churn and understanding shopping behavior. By leveraging machine learning models—especially Random Forest—we were able to accurately predict customers at risk of leaving and determine the key behavioral factors driving their decisions.

The project provides a practical foundation for targeted retention strategies. Companies can now proactively intervene with at-risk customers by offering incentives, improving support, or re-engaging them through tailored campaigns.

Overall, the insights gained from this analysis highlight the strategic importance of data-driven decision-making in e-commerce and pave the way for integrating AI-driven tools into daily business operations.

References

https://doi.org/10.1007/s10489-012-0380-5

https://doi.org/10.1509/jm.15.0414

https://doi.org/10.1016/j.sbspro.2014.09.114

https://doi.org/10.1016/j.eswa.2009.04.030

https://www.kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-prediction